UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**

Aula: 03 - Recursividade, Estruturas de Dados Lineares e Não-Lineares, Orientação a Objetos

Professor: **Raphael Martins Brum**

2026

IA PARA ENGENHEIROS
UFRGS

Da aula passada…

1) Algoritmo do Pi por Monte Carlo
   a) Incrementar o algoritmo feito em aula para realizar n rodadas de m pontos
   b) Guardar os resultados em uma lista.
   c) Calcular a média e o desvio padrão de Pi após as n rodadas, utilizando métodos próprios para as listas.

2) Algoritmos de ordenamento: Bubble Sort

3) Algoritmos de ordenamento: Insertion Sort

Aprimorar os programas para serem expressos em termos de funções

```
def funcao(arg1, arg2):
    [corpo da função]
função(var1, var2)
```

Algoritmo de Monte Carlo (versão feita em aula)

Vamos calcular o Pi pelo método de Monte Carlo. Ideia do algoritmo: é “sortear” um ponto e verificar se ele cai dentro ou fora do círculo.

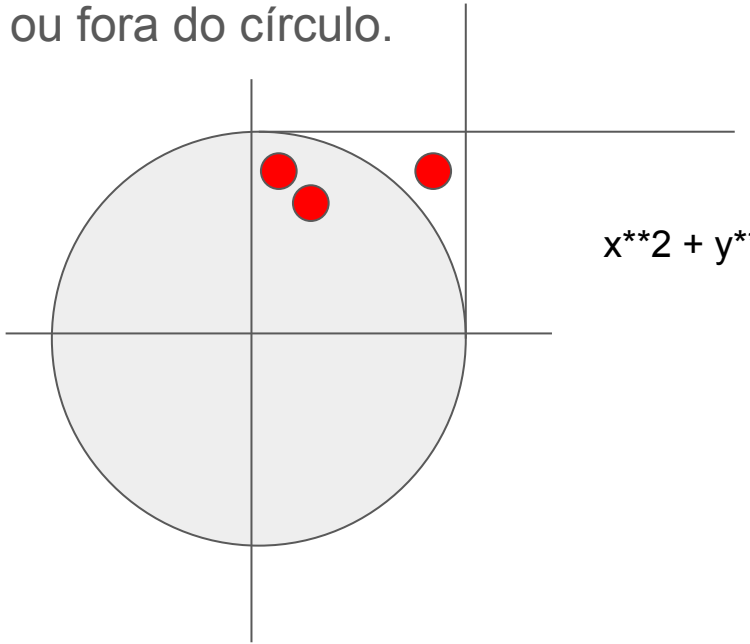
```
import random as r

pt_sorteados = 0
pt_dentro = 0
pt_a_sortear = 10000

while pt_sorteados < pt_a_sortear:
    pt = r.random(), r.random()
    if (pt[0]**2 + pt[1]**2 < 1):
        pt_dentro += 1
    pt_sorteados +=1

pi_mc = 4*pt_dentro/pt_sorteados

print(pi_mc)
```



x**2 + y**2 < 1

Algoritmo de Monte Carlo (+ função)

```python
import random as r

def pi_mc(pt_a_sortear):
    pt_sorteados = 0
    pt_dentro = 0

    while pt_sorteados < pt_a_sortear:
        pt = r.random(), r.random()
        if (pt[0]**2 + pt[1]**2 < 1):
            pt_dentro += 1
        pt_sorteados +=1

    pi_mc = 4*pt_dentro/pt_sorteados

    return(pi_mc)

print(pi_mc(10000))
```

Definimos uma função para nossa rotina de cálculo do Pi.

- A função tem um **cabeçalho** ou protótipo (header, prototype): especifica os argumentos que são recebidos.
- As variáveis locais existem apenas no escopo da função
- As variáveis do nível imediatamente superior estão disponíveis (clareza?)

Algoritmo de Monte Carlo (+estatísticas)

```python
import random as r
import numpy as np

def pi_mc(pt_a_sortear):
    pt_sorteados = 0
    pt_dentro = 0
    while pt_sorteados < pt_a_sortear:
        pt = r.random(), r.random()
        if (pt[0]**2 + pt[1]**2 < 1):
            pt_dentro += 1
        pt_sorteados +=1
    pi_mc = 4*pt_dentro/pt_sorteados
    return(pi_mc)
```

```python
rodadas = 1000
resultados = list()

while (rodadas >= 0):
        resultado = pi_mc(10000)
        resultados.append(resultado)
#       print(f"{rodadas}:
{resultado}")
        rodadas -= 1
media =
sum(resultados)/len(resultados)
desvio = np.std(resultados)
print("Media:", media)
print("Desvio:", desvio)
```

Da aula passada…

1) Algoritmo do Pi por Monte Carlo
   a) Incrementar o algoritmo feito em aula para realizar n rodadas de m pontos
   b) Guardar os resultados em uma lista.
   c) Calcular a média e o desvio padrão de Pi após as n rodadas, utilizando métodos próprios para as listas.

2) Algoritmos de ordenamento: Bubble Sort

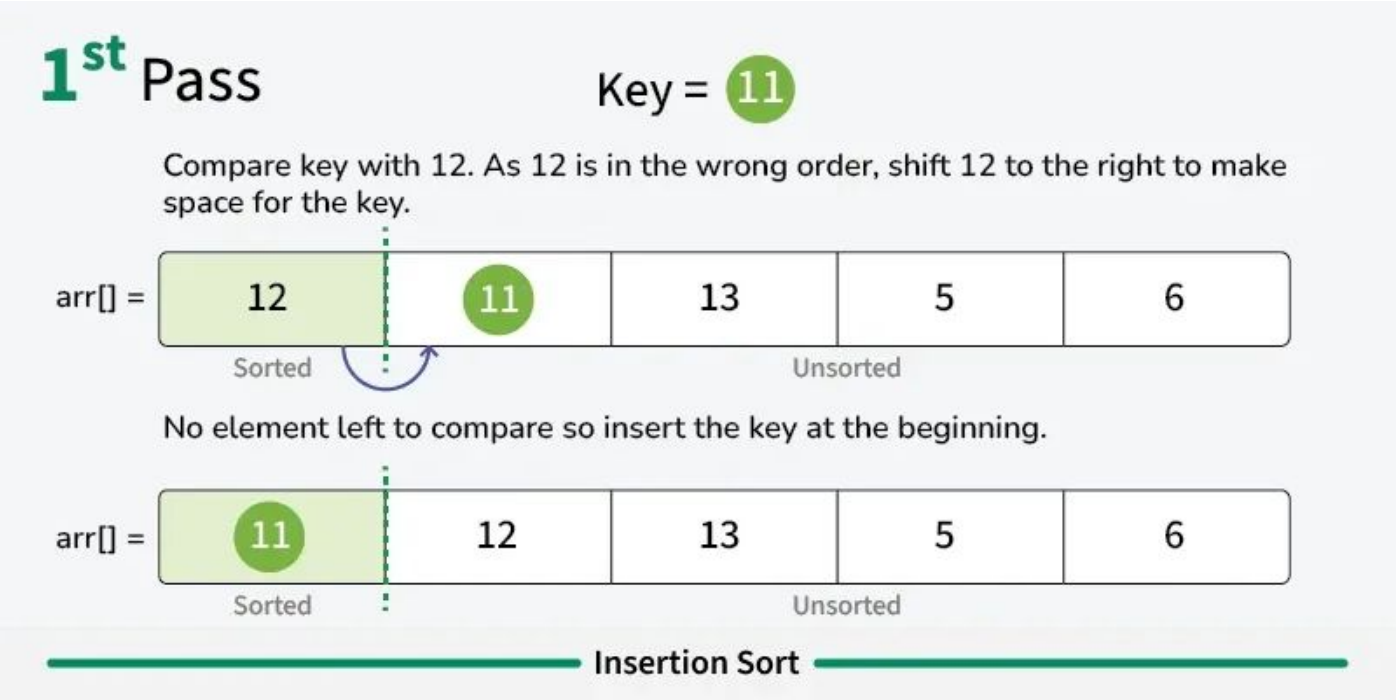
3) Algoritmos de ordenamento: Insertion Sort

Aprimorar os programas para serem expressos em termos de funções

def funcao(arg1, arg2):

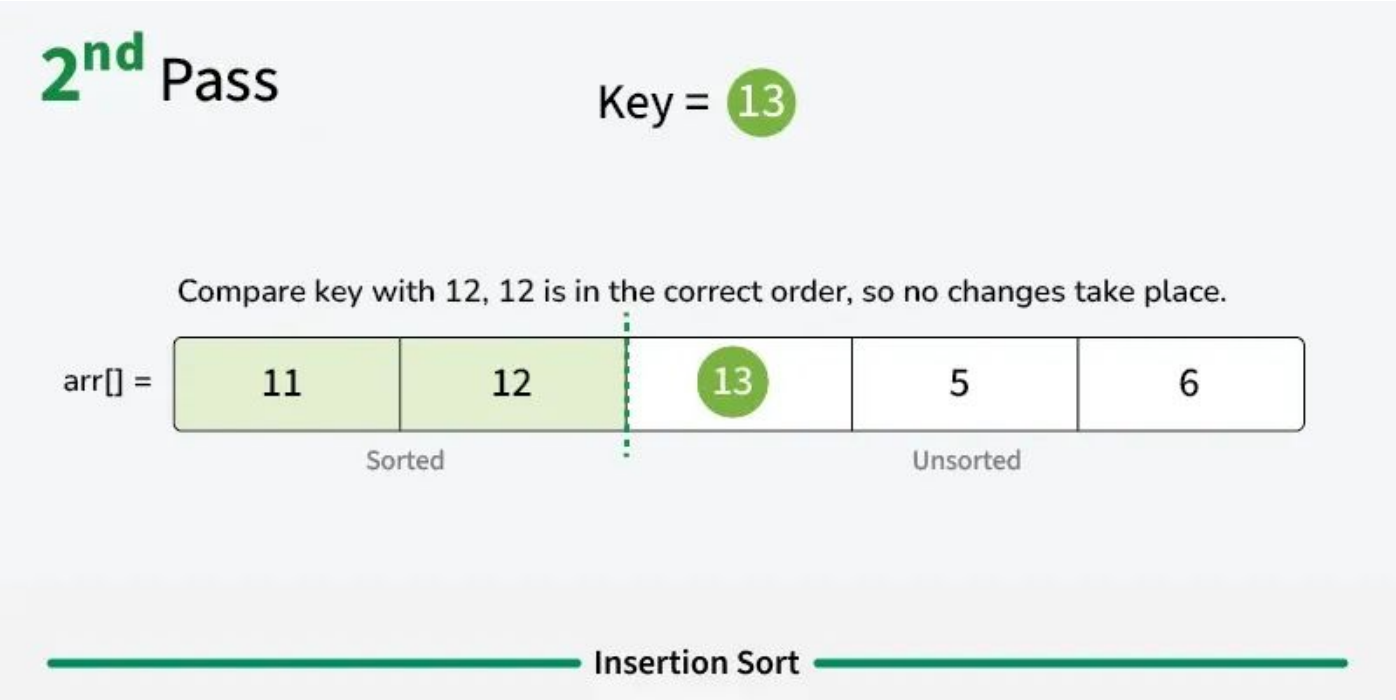
funcao(var1, var2)

Insertion Sort

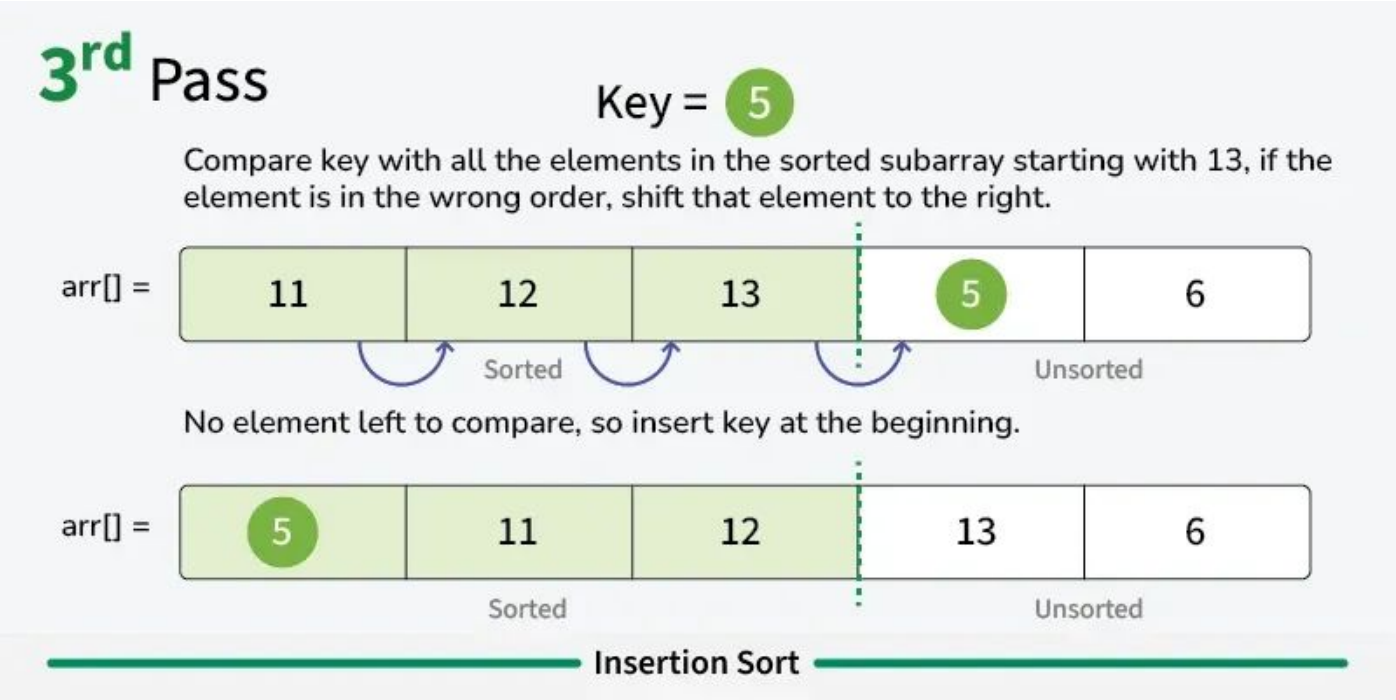


geeksforgeeks.org

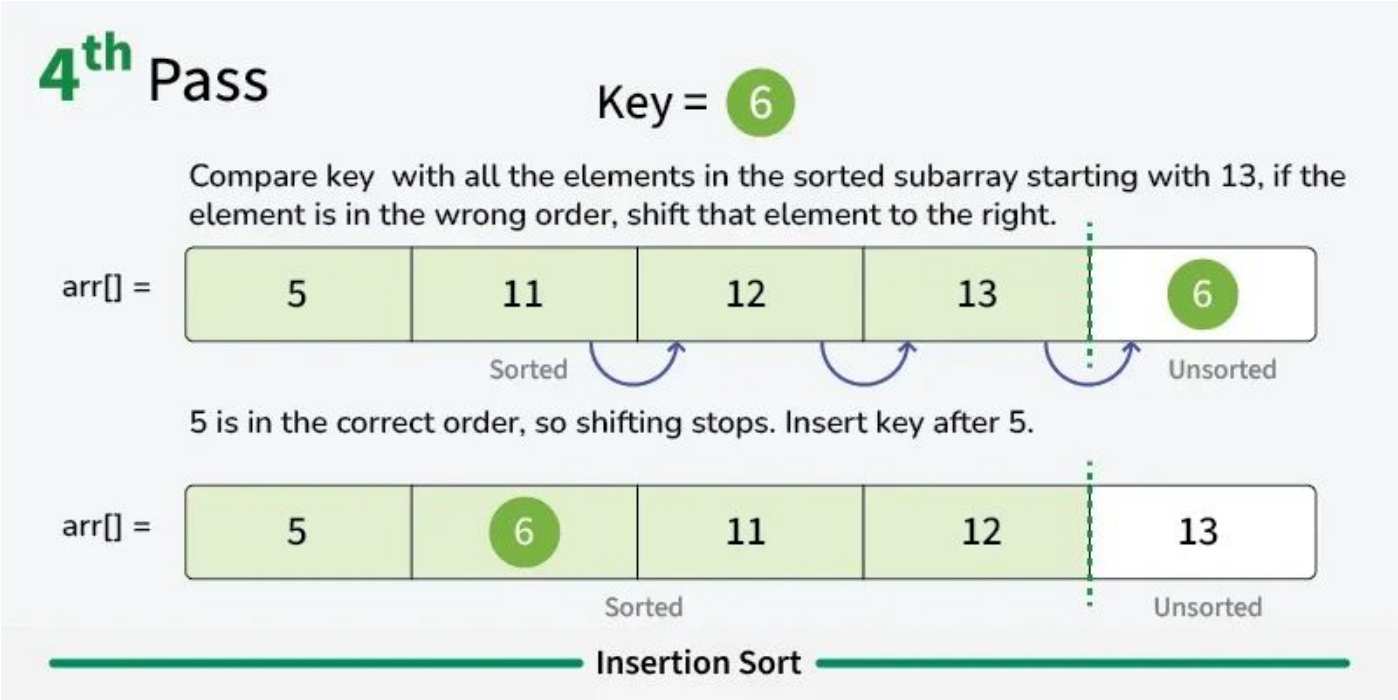
Insertion Sort



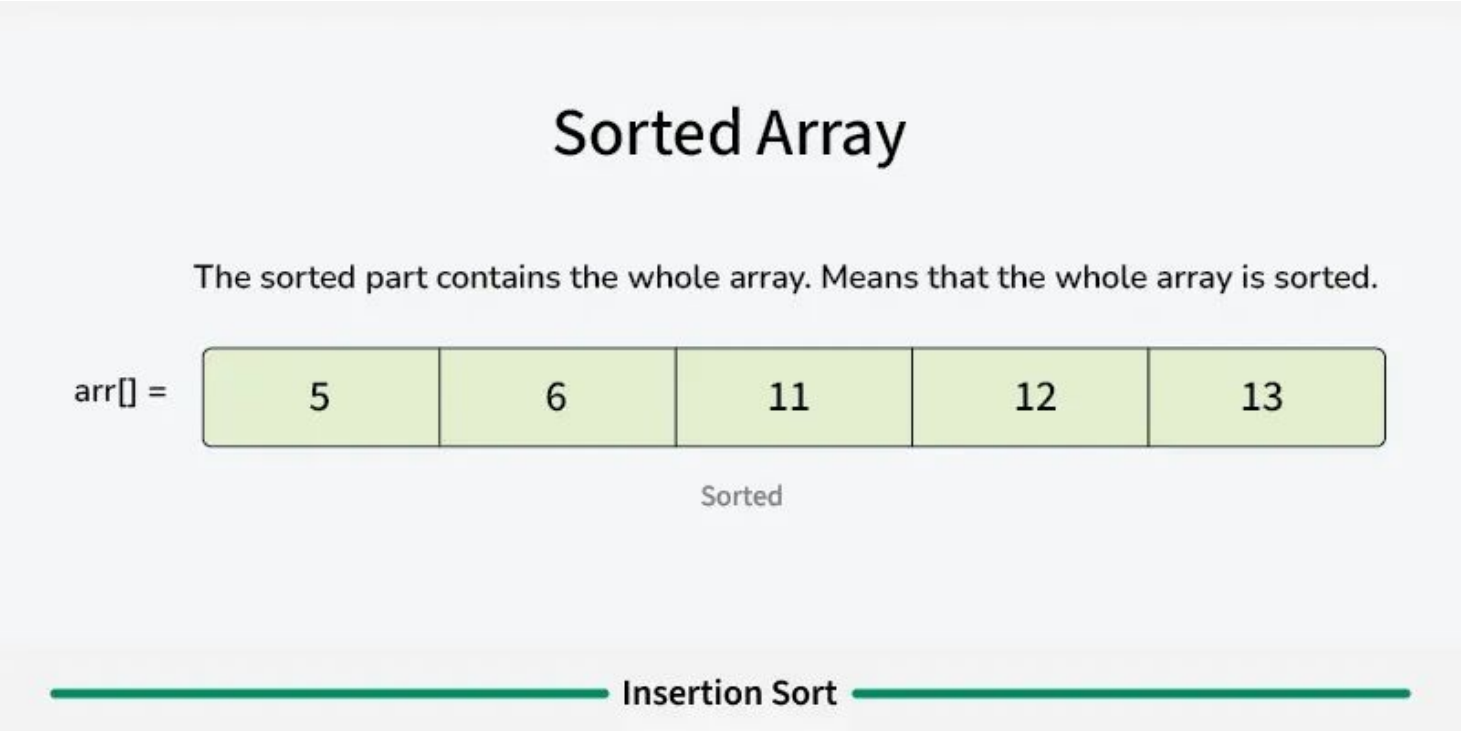
Insertion Sort



Insertion Sort



Insertion Sort



Algoritmo Insertion Sort

```python
def insertion_sort(lista):
    for i in range(1, len(lista)):
        chave = lista[i]
        j = i - 1
        while j >= 0 and chave < lista[j]:
            lista[j + 1] = lista[j]
            j -= 1
        lista[j + 1] = chave
    return lista
lista_exemplo = [23, 44, 12, 45, 11, 13, 5, 6]
print(f"Entrada: {lista_exemplo}")
lista_ordenada = insertion_sort(lista_exemplo)
print(f"Lista ordenada: {lista_ordenada}")
```

Recursão

- Processo que é definido com base em si mesmo
- Acrônimo recursivo:
  GNU = GNU is Not Unix
  Pip = Pip Installs Packages
  LAME = LAME Ain't an MP3 Encoder

- Definição precisa depender de um caso simples básico, sob pena de entrar em loop infinito

Exemplo: Fatorial

8! = 8*7*6*5*4*3*2*1*0!

7! = 7*6*5*4*3*2*1*0!

6! = 6*5*4*3*2*1*0!

(...)

2! = 2*1*0!

1! = 1*0!

0! = 1

```python
# algoritmo iterativo
def fatorial(n):
    if n == 0:
        return 1
    elif n > 0:
      fatorial = 1
      for i in range(1, n + 1):
          fatorial *= i
      return fatorial

print(fatorial(5))
```

Exemplo: Fatorial

8! = 8*7!
7! = 7*6!
6! = 6*5!
(...)
1! = 1*0!
0! = 1

Regra geral (indutiva):
n! = n*(n-1)!
Caso de base
0! = 1

```python
# algoritmo recursivo
def fatorial(n):
    if n == 0:
        return 1
    else:
        return n * fatorial(n - 1)
```

Exercício

1) Calcular a sequência de Fibonacci com um algoritmo iterativo e com um algoritmo recursivo.

Fibonacci = [0, 1, 1, 2, 3, 5, 8, …]

Exercício

1) Calcular a sequência de Fibonacci com um algoritmo iterativo e com um algoritmo recursivo.

```
# Fibonacci (recursivo)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Exercício

1) Calcular a sequência de Fibonacci com um algoritmo iterativo e com um algoritmo recursivo.

```
# Fibonacci (iterativo)

def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Tipos de Dados Estruturados

```python
user_data = {
    "user_id": 101,
    "name": "Alice",
    "roles": ["admin", "editor"],
    "preferences": {
        "theme": "dark",
        "notifications": "enabled"
    }
}

print(user_data["preferences"]["theme"])
print(user_data["roles"][0])
```

dict

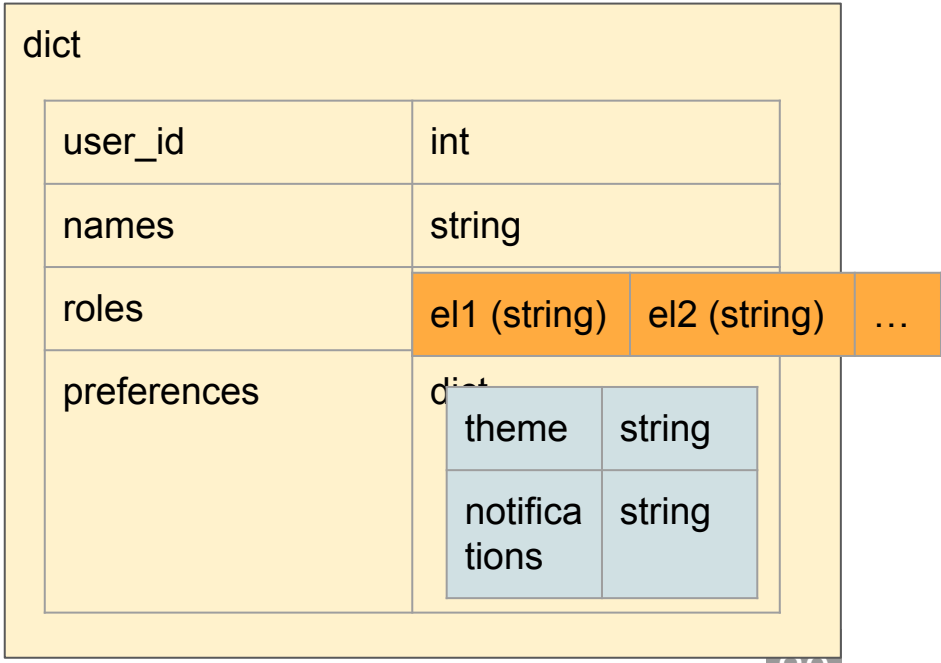
user_id	int
names	string
roles	list(string)
preferences	dict

theme	string
notifications	string

Tipos de Dados Estruturados

```python
user_data = {
    "user_id": 101,
    "name": "Alice",
    "roles": ["admin", "editor"],
    "preferences": {
        "theme": "dark",
        "notifications": "enabled"
    }
}

print(user_data["preferences"]["theme"])
print(user_data["roles"][0])
```



Objetos

```python
class Usuario:
    def __init__(self, nome): #construtor
        self.nome = nome
        self.idade = 40

    def set_name(self, nome): #setter
        self.nome = nome

    def get_name(self): #getter
        return self.nome
```

Usuario

nome	string
idade	int

.set_name(nome)
.get_name()

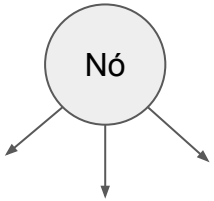
Listas, filas e outras estruturas lineares

- Listas, vetores
- Listas encadeadas (*linked lists*)
- Listas duplamente encadeadas
- Pilhas
- Filas

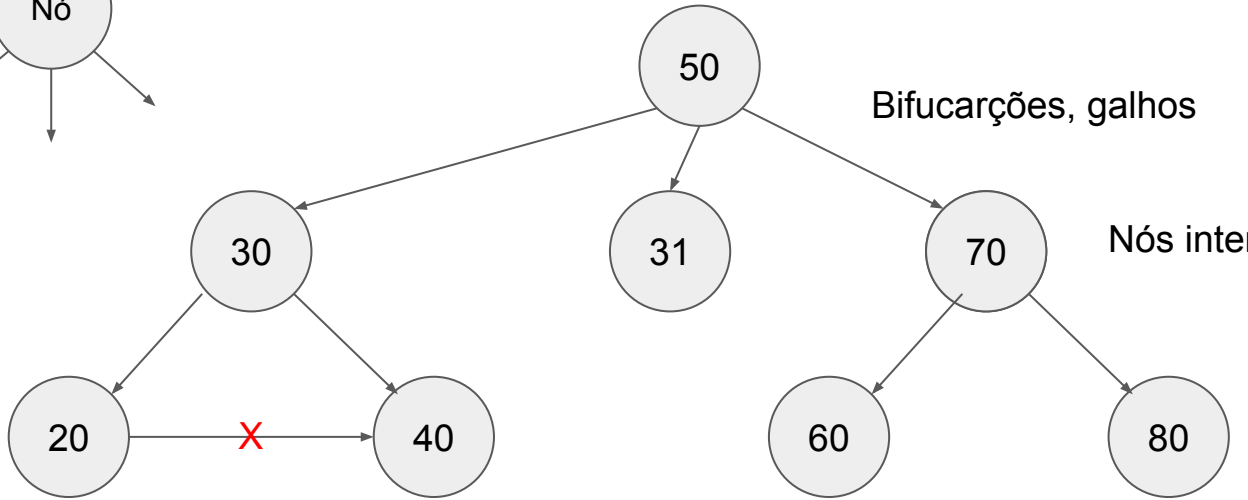
Estruturas de dados não-lineares

- **Árvores**
- Grafos

Árvore genérica



Raiz



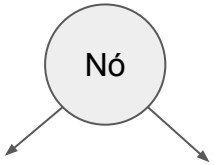
Bifucarções, galhos

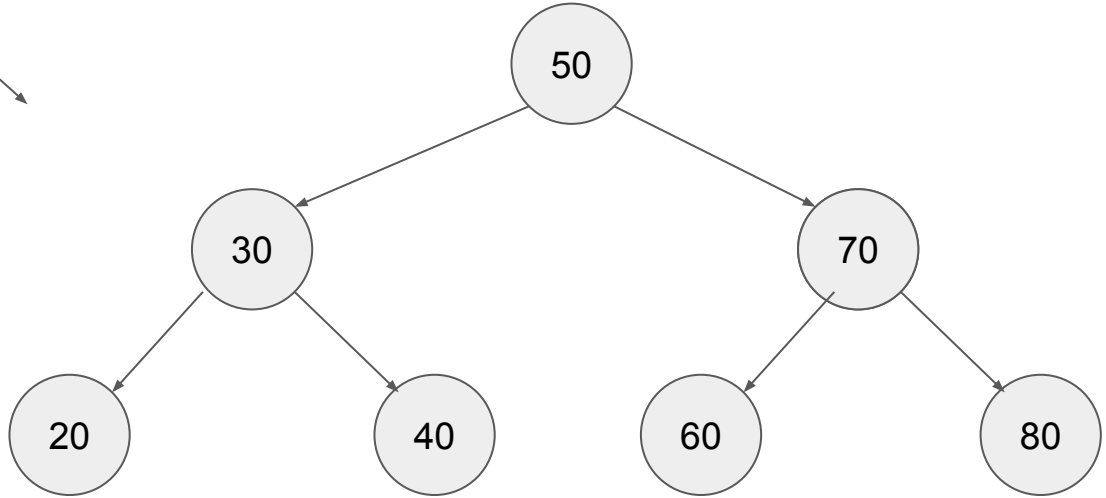
Nós intermediários

Árvores não podem ter ciclos!

Nó folha

Exemplo: árvore binária de busca





Exercício

1) Definir as classes para construirmos uma árvore binária de busca.
2) Definir métodos para inserir valores de uma lista numérica nesta árvore, respeitando a seguinte lógica:

   Valor do Filho 1 < Valor do Pai
   Valor do Filho 2 > Valor do Pai

   Precisamos que um objeto “possua” outros objetos do mesmo tipo.
   (pai irá possuir os dois filhos: filho_esq, filho_dir)

3) Implementar uma função que percorre toda a árvore na ordem, exibindo o valor dos nós na sequência em que são visitados.

4) Implementar uma função que remove um valor da árvore.

   (pensar o que fazer com os filhos!)